#1 Method `isBipartite` takes a graph, and returns `true` if it is bipartite, together with the two partitions. The argument must be a connected, non-empty graph.
#2 Initialize a simple FIFO queue.
#3 Add a random vertex of the graph to the queue, so that it will be extracted on the first iteration.
#4 Create a new hash table to keep track of the color of each vertex
#5 We (arbitrarily) pick the color red for the starting point (currently at the top of the queue).
#6 Start a loop, running until the queue is empty. It will run at least once, because of line #3.
#7 Dequeue the head of the queue (equivalently, extract the top of a priority queue). This will become the current vertex.
#8 Iterate over the outgoing edges for current vertex.
#9 If the neighbor has already been colored with the same color as v, the graph is not bipartite.
#10 If, instead, u hasn't been colored yet, we can assign it a different color and add it to the queue.
#11 The graph is bipartite: we can easily partition the vertices based on their color.

To do so, we exploit a property of bipartite graphs: a graph is bipartite if and only if it's possible to color its vertices with exactly 2 different colors, such that there isn't any pair of adjacent vertices with the same color. Figure 15.10 shows a few examples to clarify this definition.

We can perform graph coloring easily by modifying BFS algorithm: